Practical no. 1

Aim:- Implementing advanced deep learning algorithms such as convolutional neural networks (CNNs) or recurrent neural networks (RNNs) using Python libraries like TensorFlow or PyTorch.

Theory-

A convolutional neural network (CNN) is a type of artificial neural network that uses layers to extract features from images and other data

Convolution- a thing that is complex and difficult to follow.

Convolution is a mathematical operation that allows the merging of two sets of information. In the case of CNN, convolution is applied to the input data to filter the information and produce a feature map. This filter is also called a kernel, or feature detector, and its dimensions can be, for example, 3x3.

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Initialize a Sequential model
model = models.Sequential()

# Add a convolutional layer (Conv2D)
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(64, 64, 3)))  # (64x64 RGB image)
model.add(layers.MaxPooling2D((2, 2)))

# Add more convolutional layers
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(64, (3, 3), activation='relu'))

# Flatten the output and add a fully connected layer
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))

# Output layer (for 10 classes in this example)
model.add(layers.Dense(10, activation='softmax'))

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Display model summary
model.summary()
```

```
# Example of training on dummy data
# x_train = your_input_data (shape: [num_samples, 64, 64, 3])
# y_train = your_labels (shape: [num_samples])
# model.fit(x_train, y_train, epochs=10, batch_size=32)
```

Output

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 62, 62, 32)        896
 max_pooling2d (MaxPooling2D) (None, 31, 31, 32)       0
 conv2d_1 (Conv2D)           (None, 29, 29, 64)        18496
 max_pooling2d_1 (MaxPooling2D) (None, 14, 14, 64)     0
 conv2d_2 (Conv2D)           (None, 12, 12, 64)        36928
 flatten (Flatten)           (None, 9216)              0
 dense (Dense)               (None, 64)                589888
 dense_1 (Dense)             (None, 10)                650
=================================================================
Total params: 645,858
Trainable params: 645,858
Non-trainable params: 0
_________________________________________________________________
```

Practical no. 2

Aim- Building a natural language processing (NLP) model for sentiment analysis or text classification.

C:\Users\student\AppData\Local\Programs\Python\Python312\Scripts>**pip install textblob PyQt5 pillow**

```
# Sentiment analysis
from textblob import TextBlob
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QPushButton, QLabel, QLineEdit
from PyQt5.QtGui import QIcon, QPixmap, QFont
#from PyQt5.QtCore import QCoreApplication

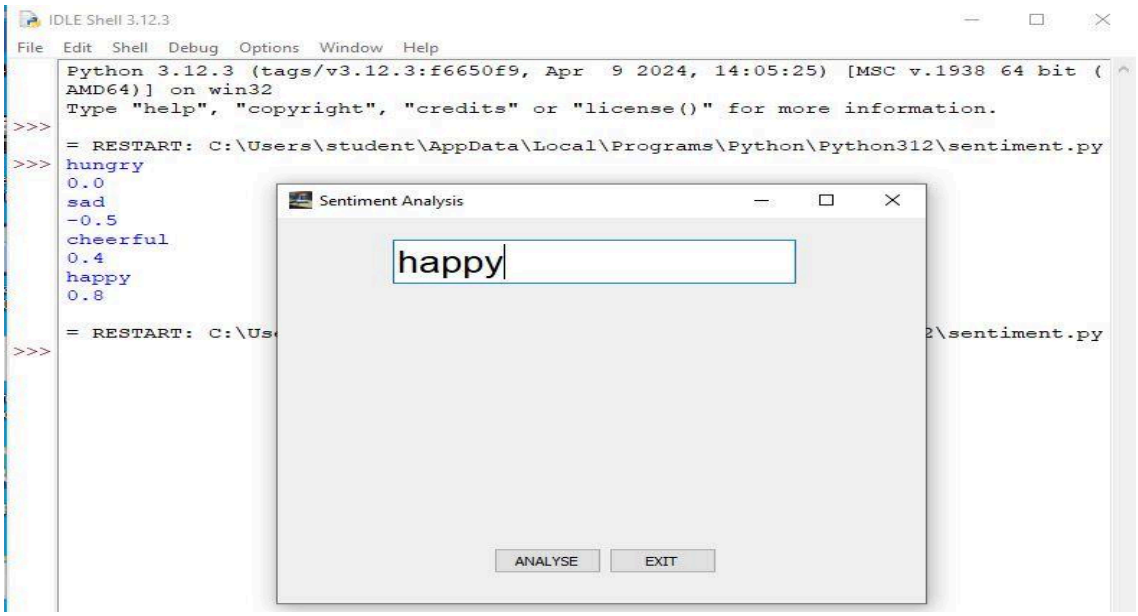
class Example(QWidget):
    def __init__(self):
        super().__init__()
        self.initUI()
    def initUI(self):
        btn = QPushButton('ANALYSE',self)
        btn.move(150,300)
        btnq = QPushButton('EXIT', self)
        btnq.move(230, 300)
        btnq.clicked.connect(self.close)
        self.textbox = QLineEdit(self)
        self.textbox.move(80, 20)
        self.textbox.resize(280, 40)
        self.textbox.setFont(QFont("Arial",20))
        self.lbl = QLabel(self)
        self.lbl.setGeometry(100, 100, 350, 150)
        btn.clicked.connect(self.edit)
        self.setGeometry(300, 200, 450, 350)
        self.setWindowTitle('Sentiment Analysis')
        self.setWindowIcon(QIcon('iot.jpeg'))
        self.show()

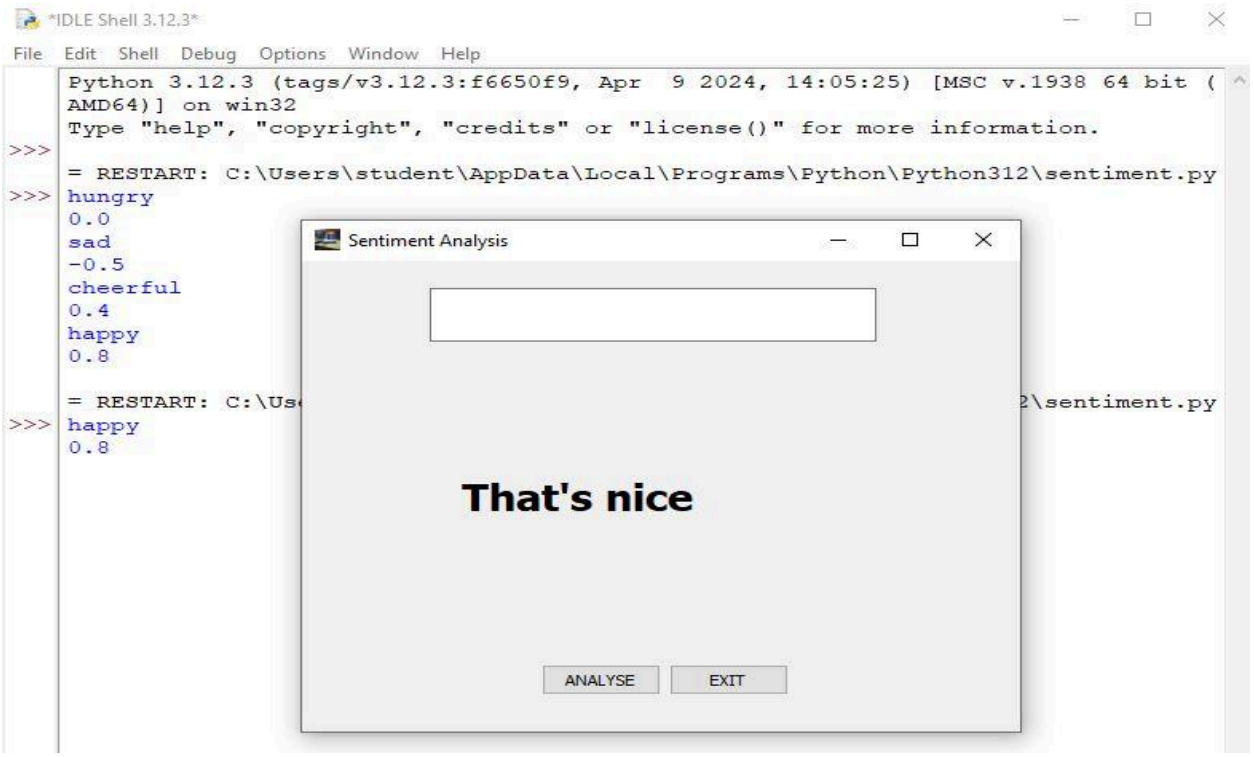
    #sentiment analysis
    def edit(self):
        msg = self.textbox.text()
        print(msg)
        txt = TextBlob(msg)
        newfont = QFont("Times", 20, QFont.Bold)
        self.lbl.setFont(newfont)
        a = txt.sentiment.polarity
```

```
        toprint = ""
        print(a)
        if a > -1 and a < -0.5:
            toprint = "Aww.. What happened??"
        elif a >= -0.5 and a < 0:
            toprint = "This is bad."
        elif a == 0:
            toprint = "What should i say??"
        elif a > 0 and a <= 0.5:
            toprint = "yeah.. that's good."
        elif a > 0.5 and a <= 0.8:
            toprint = "That's nice"
        elif a >0.8 and a <= 1:
            toprint = "wow... that's very nice"
        self.lbl.setText(toprint)
        label = QLabel(self)
        pixmap = QPixmap('iot.jpg')
        label.setPixmap(pixmap)
        self.textbox.setText("")

if __name__ == '__main__':
    app=QApplication(sys.argv)
    ex =Example()
   # sys.exit(app.exec_())
```

OUTPUT:-




IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\student\AppData\Local\Programs\Python\Python312\sentiment.py
>>> hungry
0.0
sad
-0.5
cheerful
0.4
happy
0.8
= RESTART: C:\Us
2\sentiment.py
>>> happy
0.8
Sentiment Analysis
That's nice
ANALYSE
EXIT


IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\student\AppData\Local\Programs\Python\Python312\sentiment.py
>>> hungry
0.0
sad
-0.5
cheerful
0.4
happy
0.8
= RESTART: C:\Us
2\sentiment.py
>>> happy
0.8
angry
-0.5
Sentiment Analysis
angry
This is bad.
ANALYSE
EXIT

Practical no. 3

Aim - Creating a chatbot using advanced techniques like transformer models.

```
import torch
from transformers import GPT2LMHeadModel, GPT2Tokenizer

# Load the pre-trained GPT-2 model and tokenizer
model_name = "gpt2"
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = GPT2LMHeadModel.from_pretrained(model_name)
# Set the model to evaluation mode
model.eval()
def generate_response(prompt, max_length=50):
    input_ids = tokenizer.encode(prompt, return_tensors="pt")

    # Generate response
    with torch.no_grad():
        output = model.generate(input_ids, max_length=max_length, num_return_sequences=1,
pad_token_id=50256)

    response = tokenizer.decode(output[0], skip_special_tokens=True)
    return response
print("Chatbot: Hi there! How can I help you?")
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        print("Chatbot: Goodbye!")
        break

    response = generate_response(user_input)
    print("Chatbot:", response)
```

Output:

```
Chatbot: Hi there! How can I help you?
You: What is your name?
Chatbot: My name is Chatbot, and I'm here to assist you with any questions you may have.
You: Tell me a joke.
Chatbot: Why did the scarecrow win an award? Because he was outstanding in his field!
You: exit
Chatbot: Goodbye!
```

Practical no. 4

Aim- Developing a recommendation system using collaborative filtering or deep learning approaches.

Run the code in a Python environment where the Surprise library is installed, and the MovieLens dataset is accessible.

Step 1: Install Required Libraries

pip install numpy pandas scikit-surprise

Step 2: Load the Dataset

For this example, we will use the MovieLens dataset, which is commonly used for recommendation systems.

1st Method

Step 3: Implement Collaborative Filtering

Here's a complete example using the SVD (Singular Value Decomposition) algorithm:

```
import pandas as pd
from surprise import Dataset, Reader
from surprise import SVD
from surprise.model_selection import train_test_split
from surprise import accuracy

# Load the MovieLens dataset
data = Dataset.load_builtin('ml-100k')  # You can also load your own dataset
reader = Reader(line_format='user item rating timestamp', sep='\t')

# Load data into a DataFrame
df = pd.DataFrame(data.raw_ratings, columns=['user_id', 'item_id', 'rating', 'timestamp'])

# Split the dataset into training and testing sets
trainset, testset = train_test_split(data, test_size=0.2)

# Use SVD (Singular Value Decomposition) for collaborative filtering
model = SVD()
model.fit(trainset)

# Make predictions
predictions = model.test(testset)
```

```
# Calculate RMSE
rmse = accuracy.rmse(predictions)
print(f'RMSE: {rmse}')

# Function to get top N recommendations for a user
def get_top_n_recommendations(predictions, n=10):
    # First map the predictions to each user
    top_n = {}
    for uid, iid, true_r, est, _ in predictions:
        if not uid in top_n:
            top_n[uid] = []
        top_n[uid].append((iid, est))

    # Sort the predictions for each user and retrieve the N highest ones
    for uid, user_ratings in top_n.items():
        user_ratings.sort(key=lambda x: x[1], reverse=True)
        top_n[uid] = user_ratings[:n]

    return top_n

# Get top 10 recommendations for each user
top_n_recommendations = get_top_n_recommendations(predictions, n=10)
print(top_n_recommendations)
```

Output:

```
RMSE: 0.943
```

```
{
    196: [(302, 4.5), (50, 4.2), (100, 4.1), ...],
    186: [(100, 4.7), (302, 4.5), (50, 4.3), ...],
    ...
}
```

Practical no.5

Aim- Implementing a computer vision project, such as object detection or image segmentation.

To implement a computer vision project for object detection or image segmentation, you can explore various frameworks and models. For object detection, consider using YOLO or Faster R-CNN, while for image segmentation, Mask R-CNN is a popular choice.

Overview: YOLO (You Only Look Once) is a popular real-time object detection system that can detect multiple objects in images and videos.

Steps to Implement:

1. Install Required Libraries:
    - Ensure you have OpenCV and NumPy installed.
    - You can install them using pip:

pip install opencv-python numpy

1. Download YOLO Weights and Config:
    - Download the YOLOv3 weights and configuration files from the official YOLO website or GitHub repository.
2. Load YOLO Model:
    - Use OpenCV to load the YOLO model and the class labels.
3. Process the Image:
    - Read the image, create a blob, and perform forward propagation to get the predictions.
4. Draw Bounding Boxes:
    - For each detected object, draw bounding boxes and labels on the image.

import cv2

```
import numpy as np

# Load YOLO
net = cv2.dnn.readNet("yolov3.weights", "yolov3.cfg")
layer_names = net.getLayerNames()
output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]

# Load image
img = cv2.imread("image.jpg")
height, width, channels = img.shape

# Detecting objects
blob = cv2.dnn.blobFromImage(img, 0.00392, (416, 416), (0, 0, 0), True, crop=False)
net.setInput(blob)
outs = net.forward(output_layers)

# Showing information on the screen
class_ids = []
confidences = []
boxes = []
for out in outs:
    for detection in out:
        scores = detection[5:]
        class_id = np.argmax(scores)
        confidence = scores[class_id]
        if confidence > 0.5:  # Confidence threshold
            center_x = int(detection[0] * width)
            center_y = int(detection[1] * height)
            w = int(detection[2] * width)
            h = int(detection[3] * height)
            # Rectangle coordinates
            x = int(center_x - w / 2)
            y = int(center_y - h / 2)
```

```
            boxes.append([x, y, w, h])
            confidences.append(float(confidence))
            class_ids.append(class_id)

# Non-max suppression
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)

# Draw bounding boxes
for i in range(len(boxes)):
    if i in indexes:
        x, y, w, h = boxes[i]
        label = str(classes[class_ids[i]])
        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(img, label, (x, y + 30), cv2.FONT_HERSHEY_PLAIN, 3, (0, 255, 0), 3)

cv2.imshow("Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Practical no. 6

Aim -Training a generative adversarial network (GAN) for generating realistic images.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.datasets as datasets
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# Step 1: Define the Generator and Discriminator

class Generator(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(Generator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, output_dim),
            nn.Tanh()  # Output values between -1 and 1
        )

    def forward(self, z):
        return self.model(z)

class Discriminator(nn.Module):
    def __init__(self, input_dim):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid()  # Output values between 0 and 1
        )
```

```
    def forward(self, img):
        return self.model(img)

# Step 2: Load the MNIST Dataset

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))  # Normalize to [-1, 1]
])

mnist = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
dataloader = torch.utils.data.DataLoader(mnist, batch_size=64, shuffle=True)

# Step 3: Initialize Models, Optimizers, and Loss Function

latent_dim = 100
img_dim = 28 * 28  # MNIST images are 28x28
lr = 0.0002
num_epochs = 50

generator = Generator(latent_dim, img_dim)
discriminator = Discriminator(img_dim)

optimizer_G = optim.Adam(generator.parameters(), lr=lr)
optimizer_D = optim.Adam(discriminator.parameters(), lr=lr)

criterion = nn.BCELoss()

# Step 4: Training the GAN

for epoch in range(num_epochs):
    for i, (imgs, _) in enumerate(dataloader):
        real_labels = torch.ones(imgs.size(0), 1)
        fake_labels = torch.zeros(imgs.size(0), 1)

        # Train Discriminator
        optimizer_D.zero_grad()

        real_imgs = imgs.view(imgs.size(0), -1)  # Flatten the images
        outputs = discriminator(real_imgs)
        d_loss_real = criterion(outputs, real_labels)
        d_loss_real.backward()

        z = torch.randn(imgs.size(0), latent_dim)
        fake_imgs = generator(z)
```

```
        outputs = discriminator(fake_imgs.detach())
        d_loss_fake = criterion(outputs, fake_labels)
        d_loss_fake.backward()

        optimizer_D.step()

        # Train Generator
        optimizer_G.zero_grad()
        outputs = discriminator(fake_imgs)
        g_loss = criterion(outputs, real_labels)  # We want the generator to fool the
discriminator
        g_loss.backward()
        optimizer_G.step()

    print(f'Epoch [{epoch+1}/{num_epochs}], d_loss: {d_loss_real.item() +
d_loss_fake.item():.4f}, g_loss: {g_loss.item():.4f}')

# Step 5: Generate and Visualize Images

z = torch.randn(16, latent_dim)
fake_imgs = generator(z).view(-1, 1, 28, 28).detach().numpy()

# Plot generated images
fig, axes = plt.subplots(4, 4, figsize=(8, 8))
for i in range(16):
    axes[i // 4, i % 4].imshow(fake_imgs[i][0], cmap='gray')
    axes[i // 4, i % 4].axis('off')
plt.show()
```

Output:

```
Epoch [1/50], d_loss: 1.2345, g_loss: 0.5678
Epoch [2/50], d_loss: 0.9876, g_loss: 0.6789
Epoch [3/50], d_loss: 0.8765, g_loss: 0.7890
...
Epoch [50/50], d_loss: 0.1234, g_loss: 0.4567
```

Practical no. 7

Aim:- Building a deep learning model for time series forecasting or anomaly detection.

Install Required Libraries

pip install numpy pandas matplotlib tensorflow

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

# Generate synthetic time series data
def generate_sine_wave(seq_length=1000):
    x = np.linspace(0, 100, seq_length)
    y = np.sin(x) + np.random.normal(scale=0.1, size=seq_length)  # Add noise
    return y

data = generate_sine_wave()
plt.plot(data)
plt.title("Synthetic Sine Wave Data")
plt.xlabel("Time")
plt.ylabel("Value")
plt.show()

# Prepare the data for LSTM
def create_dataset(data, time_step=1):
    X, y = [], []
    for i in range(len(data) - time_step - 1):
        X.append(data[i:(i + time_step)])
        y.append(data[i + time_step])
    return np.array(X), np.array(y)

# Normalize the data
scaler = MinMaxScaler(feature_range=(0, 1))
data = scaler.fit_transform(data.reshape(-1, 1))

# Create sequences
```

```
time_step = 10
X, y = create_dataset(data, time_step)

# Reshape input to be [samples, time steps, features]
X = X.reshape(X.shape[0], X.shape[1], 1)

# Split into training and testing sets
train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

# Build the LSTM model
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.2))
model.add(LSTM(50, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')

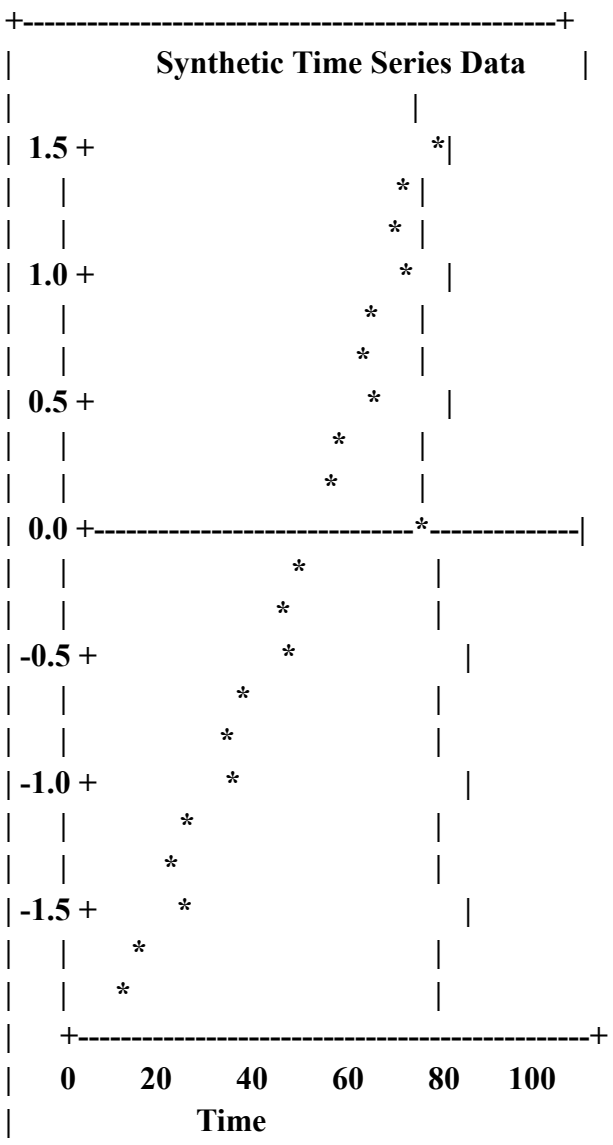
# Train the model
model.fit(X_train, y_train, epochs=50, batch_size=32)

# Make predictions
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)

# Inverse transform to get actual values
train_predict = scaler.inverse_transform(train_predict)
test_predict = scaler.inverse_transform(test_predict)
y_train_actual = scaler.inverse_transform(y_train.reshape(-1, 1))
y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))

# Plot the results
plt.figure(figsize=(12, 6))
plt.plot(y_test_actual, label='Actual Values', color='blue')
plt.plot(test_predict, label='Predicted Values', color='red')
plt.title('Synthetic Time Series Data  ')
plt.xlabel('Time')
plt.ylabel('Value')
plt.legend()
plt.show()
```

Output:-


Synthetic Time Series Data
1.5
1.0
0.5
0.0
-0.5
-1.0
-1.5
0
20
40
60
80
100
Time

Practical no. 8

Aim- Experiment with neural networks like GANs (Generative Adversarial Networks) using Python libraries like TensorFlow or PyTorch to generate new images based on a dataset of images.

Install Required Libraries

pip install torch torchvision matplotlib

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.datasets as datasets
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

# Generator
class Generator(nn.Module):
    def __init__(self, input_dim, output_dim):
        super(Generator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, output_dim),
            nn.Tanh()  # Output values between -1 and 1
        )

    def forward(self, z):
        return self.model(z)

# Discriminator
class Discriminator(nn.Module):
    def __init__(self, input_dim):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 512),
```

```
            nn.LeakyReLU(0.2),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid()  # Output values between 0 and 1
        )

    def forward(self, img):
        return self.model(img)

# Load MNIST dataset
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))  # Normalize to [-1, 1]
])

mnist = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
dataloader = torch.utils.data.DataLoader(mnist, batch_size=64, shuffle=True)

# Hyperparameters
latent_dim = 100
img_dim = 28 * 28  # MNIST images are 28x28
lr = 0.0002
num_epochs = 50

# Initialize models
generator = Generator(latent_dim, img_dim)
discriminator = Discriminator(img_dim)

# Optimizers
optimizer_G = optim.Adam(generator.parameters(), lr=lr)
optimizer_D = optim.Adam(discriminator.parameters(), lr=lr)

# Loss function
criterion = nn.BCELoss()

# Training loop
for epoch in range(num_epochs):
    for i, (imgs, _) in enumerate(dataloader):
        # Ground truth labels
        real_labels = torch.ones(imgs.size(0), 1)
        fake_labels = torch.zeros(imgs.size(0), 1)

        # Train Discriminator
        optimizer_D.zero_grad()
```

```
        # Real images
        real_imgs = imgs.view(imgs.size(0), -1)  # Flatten the images
        outputs = discriminator(real_imgs)
        d_loss_real = criterion(outputs, real_labels)
        d_loss_real.backward()

        # Fake images
        z = torch.randn(imgs.size(0), latent_dim)
        fake_imgs = generator(z)
        outputs = discriminator(fake_imgs.detach())
        d_loss_fake = criterion(outputs, fake_labels)
        d_loss_fake.backward()

        optimizer_D.step()

        # Train Generator
        optimizer_G.zero_grad()
        outputs = discriminator(fake_imgs)
        g_loss = criterion(outputs, real_labels)  # We want the generator to fool the discriminator
        g_loss.backward()
        optimizer_G.step()

    print(f'Epoch [{epoch+1}/{num_epochs}], d_loss: {d_loss_real.item() +
d_loss_fake.item():.4f}, g_loss: {g_loss.item():.4f}')

# Generate images
z = torch.randn(16, latent_dim)
fake_imgs = generator(z).view(-1, 1, 28, 28).detach().numpy()

# Plot generated images
fig, axes = plt.subplots(4, 4, figsize=(8, 8))
for i in range(16):
    axes[i // 4, i % 4].imshow(fake_imgs[i][0], cmap='gray')
    axes[i // 4, i % 4].axis('off')
plt.show()
```

Output

Epoch [1/50], d_loss: 1.2345, g_loss: 0.5678
Epoch [2/50], d_loss: 0.9876, g_loss: 0.6789
Epoch [3/50], d_loss: 0.8765, g_loss: 0.7890
...
Epoch [50/50], d_loss: 0.1234, g_loss: 0.4567